# CS456 – Modern Cybersecurity

Mini Group Project 1: Resilient Security Systems – Part 2 (Recovery)

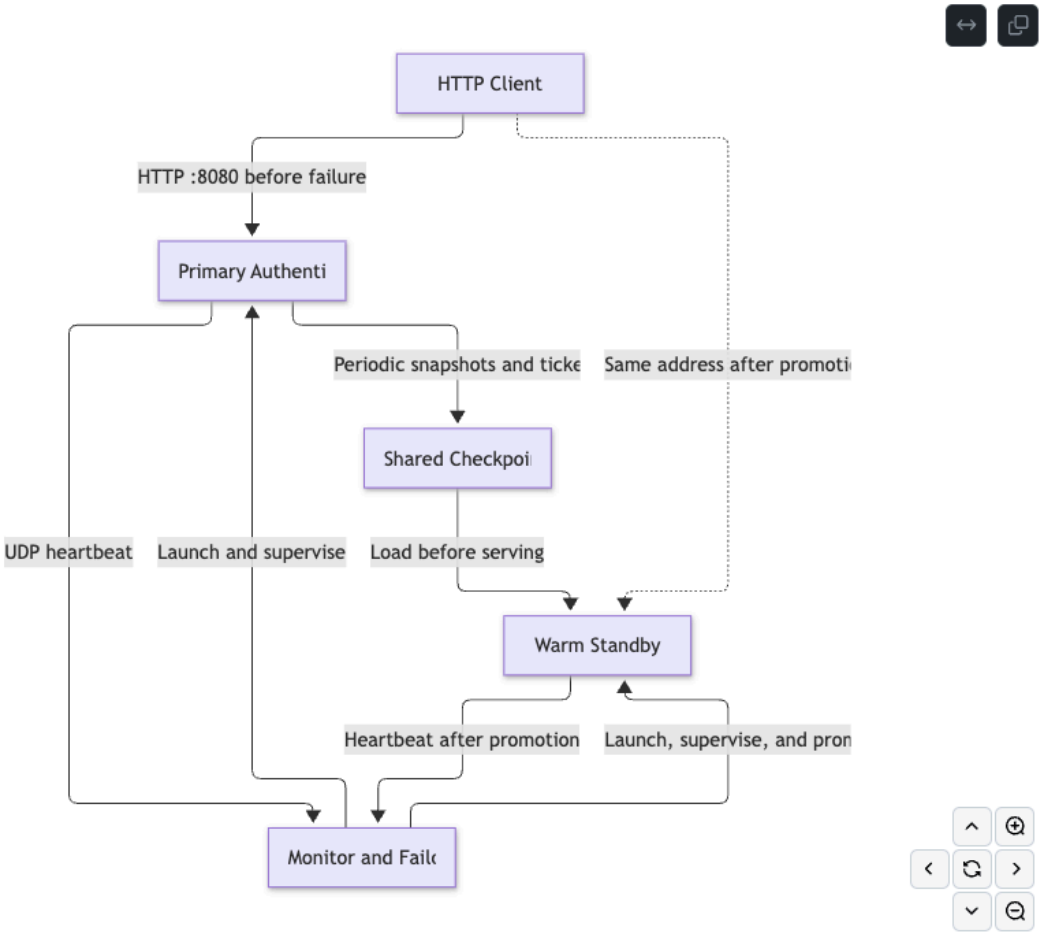
**Project Report**

**Project Repository:** https://github.com/ybenchouaf/cs456gp1

**Team Members:**

- Youssef Benchouaf
- Mark Robinson
- Jane Rassadina
- Nadya Radaeva

**Software Architectural Diagram**



**Redundancy Justification**

Our team selected a passive redundancy with a warm standby.
The primary handles authentication requests, while the standby runs as a waiting process until the controller promotes it. Session state is transferred through a local checkpoint file.

This approach reuses the Part 1 authentication API and heartbeat mechanism while keeping replication and client routing simple.

The trade-offs are checkpoint overhead and a nonzero recovery delay. Checkpointing before a successful login response preserves acknowledged ticket issuance across the simulated primary process crash.

**Execution & Setup Instructions**

1. Prerequisites
    a. Python 3.11 or later
    b. VS Code or another terminal environment
    c. Project source code
2. Environment Setup
    a. Open a terminal in the project root directory
    b. Create and activate a virtual environment (macOs/Linux)

        i. 
```
python -m venv .venv
source .venv/bin/activate
```

3. Start the Failover System by running command in the first terminal:

    a. 
```
python -m part2.monitor
```

    b. Note: this would automatically launch two instances:
        i. Primary - for authentication requests, checkpoints, UDP heartbeats
        ii. Secondary - waiting for promotion command and picks up after primary failure
4. Test the authentication API
    a. The system retains the Part 1 demonstration credentials:

i. **Username: demo**
ii. **Password: password123**

iii. **POST /login**

1. **Valid credentials return a random ticket and** expires_in: 60 seconds**.**

b. Open a second terminal and run login command:

```
curl -X POST http://127.0.0.1:8080/login \
-H "Content-Type: application/json" \
-d '{"username":"demo","password":"password123"}'
```

c. **Check the service health:**

i. **GET /health**

1. **A before failover service returns HTTP 200 with:**

**a.** {
**b.** "status": "ready",
**c.** "node_id": "**primary**",
**d.** "pid": 12345
**e.** }

2. **After failover service returns HTTP 200 with:**

**a.** {
**b.** "status": "ready",
**c.** "node_id": "**secondary**",
**d.** "pid": 67890
**e.** }

3.

ii. Run command:

```
curl http://127.0.0.1:8080/health
```

d. **Validate the ticket:**

i. **POST /validate** accepts a ticket and returns: {"valid": true} **or:** {"valid": false}

ii. Run command:

```
curl -X POST http://127.0.0.1:8080/validate \
-H "Content-Type: application/json" \
-d '{"ticket":"YOUR_TICKET_HERE"}'
```

5. Authentication Failover Testing
    1. The controller starts the primary and secondary services.
    2. The primary serves requests, saves checkpoints, and sends heartbeats.
    3. The primary crashes after a random 20–40 second delay.
    4. The controller reaches the heartbeat timeout and checks its primary process handle.
    5. If the primary has exited, the controller logs [Crash Detected].
    6. It logs [Failover Triggered] and sends the promotion command once.
    7. The standby loads the checkpoint and starts serving.
    8. The controller verifies secondary readiness and logs [Service Restored].

Recovery events are timestamped and written to the incident log. Startup and readiness checks have bounded waits. Failed recovery produces an explicit error rather than a restoration message. Stopping the controller cleans up its managed child processes.

**Adversarial Audit Summary**

If heartbeats stop while the primary process is still running, the controller:

- Logs the disruption.
- Keeps the secondary service in standby mode.
- Continues monitoring the primary process state.
- If the primary later exits, completes the recovery sequence after the timeout condition is satisfied.

This local process-exit policy avoids promotion based solely on missing heartbeats. It does not recover alive but hangs primarily.

The corrupted-checkpoint audit must arrange for corruption to remain present until the standby attempts to load it. A subsequent primary snapshot must not silently replace the audit payload before that attempt.

Malformed checkpoint recovery must fail explicitly. Structural validation does not authenticate valid-looking malicious modifications.